马来西亚留学生汉语练习平台源代码

软件版本：V1.0

马来西亚留学生汉语练习平台源代码

1 源代码属性结构说明

本源码稿由后台服务、前台 H5 页面、数据库脚本和项目配置组成。后台 admin 使用 Node.js、Express、mysql2、JWT、bcryptjs 实现认证、练习、管理、日志和数据库事务；前台 fronter 使用 Vue 3、Vue Router、Vite、CSS 和原生状态对象实现学生端、管理端和三语界面；数据库脚本创建并初始化用户、等级、分类、题库、试卷、成绩、错题和翻译数据。

源码稿排除了 node_modules、dist、运行日志、浏览器缓存和第三方依赖源码。涉及密码、令牌、邮箱、主机地址和数据库口令的内容均在输出前进行了脱敏处理。为提高审阅可读性，源码正文按文件逐项列出，保留真实相对路径、文件职责、所属模块和带行号的源码正文。

2 目录与模块结构

文件路径	所属层次	文件职责
admin/src/server.js	后台/数据层	项目运行、构建或说明配置。
admin/src/app.js	后台/数据层	项目运行、构建或说明配置。
admin/src/config/db.js	后台/数据层	MySQL 连接池配置和事务封装。
admin/src/middleware/auth.js	后台/数据层	JWT 解析和登录态校验中间件。
admin/src/middleware/role.js	后台/数据层	管理员角色授权中间件。
admin/src/middleware/requestLogger.js	后台/数据层	HTTP 请求状态、耗时和用户标识日志中间件。
admin/src/routes/auth.js	后台/数据层	注册、登录、个人资料查询和资料更新接口。
admin/src/routes/learning.js	后台/数据层	等级、分类、练习列表、试卷详情、答题提交、成绩记录和错题本接口。
admin/src/routes/admin.js	后台/数据层	管理台统计、用户列表、题库列表、试卷列表和成绩记录列表接口。
admin/src/utils/errors.js	后台/数据层	项目运行、构建或说明配置。
admin/src/utils/response.js	后台/数据层	项目运行、构建或说明配置。
admin/src/utils/logger.js	后台/数据层	日志文件写入、控制台输出和敏感字段脱敏工具。
admin/src/utils/initDb.js	后台/数据层	项目运行、构建或说明配置。
admin/src/utils/seedUsers.js	后台/数据层	项目运行、构建或说明配置。
admin/sql/schema.sql	后台/数据层	数据库和业务表结构定义脚本。
admin/sql/seed.sql	后台/数据层	等级、分类、试卷、题目、选项和三语内容初始化脚本。
fronter/src/main.js	前台页面层	项目运行、构建或说明配置。
fronter/src/App.vue	前台页面层	Vue 前台或管理端页面组件。
fronter/src/router/index.js	前台页面层	前台路由和权限守卫定义。
fronter/src/api/client.js	前台页面层	HTTP 请求、JSON 载荷和 Authorization 请求头封装。
fronter/src/stores/auth.js	前台页面层	项目运行、构建或说明配置。
fronter/src/i18n/index.js	前台页面层	三语界面文案和语言状态维护。
fronter/src/views/Home.vue	前台页面层	Vue 前台或管理端页面组件。
fronter/src/views/Login.vue	前台页面层	Vue 前台或管理端页面组件。
fronter/src/views/Register.vue	前台页面层	Vue 前台或管理端页面组件。
fronter/src/views/PracticeList.vue	前台页面层	Vue 前台或管理端页面组件。
fronter/src/views/PracticeDetail.vue	前台页面层	Vue 前台或管理端页面组件。
fronter/src/views/Records.vue	前台页面层	Vue 前台或管理端页面组件。
fronter/src/views/WrongBook.vue	前台页面层	Vue 前台或管理端页面组件。
fronter/src/views/Profile.vue	前台页面层	Vue 前台或管理端页面组件。
fronter/src/views/Dashboard.vue	前台页面层	Vue 前台或管理端页面组件。
fronter/src/views/AdminTable.vue	前台页面层	Vue 前台或管理端页面组件。
fronter/src/assets/style.css	前台页面层	前台页面通用样式。
admin/package.json	后台/数据层	项目运行、构建或说明配置。
fronter/package.json	前台页面层	项目运行、构建或说明配置。
.gitignore	项目配置	项目运行、构建或说明配置。

文件路径	所属层次	文件职责
README.md	项目配置	项目运行、构建或说明配置。

3 源码正文

文件：admin/src/server.js

> 职责：项目运行、构建或说明配置。
> 模块：后台服务与数据层

```
import app from './app.js';
import { logger } from './utils/logger.js';

const port = Number(process.env.PORT || 3000);
app.listen(port, () => {
  logger.info(`Chinese practice API running at http://localhost:${port}`);
});
```

文件：admin/src/app.js

> 职责：项目运行、构建或说明配置。
> 模块：后台服务与数据层

```
import express from 'express';
import cors from 'cors';
import dotenv from 'dotenv';
import authRoutes from './routes/auth.js';
import learningRoutes from './routes/learning.js';
import adminRoutes from './routes/admin.js';
import { requestLogger } from './middleware/requestLogger.js';
import { logger } from './utils/logger.js';

dotenv.config();

const app = express();
app.use(cors());
app.use(express.json());
app.use(requestLogger);

app.get('/api/health', (_req, res) => res.json({ code: 200, message: 'success', data: { status: 'ok' } }));
app.use('/api/auth', authRoutes);
app.use('/api/learning', learningRoutes);
app.use('/api/admin', adminRoutes);

app.use((req, res) => res.status(404).json({ code: 404, message: `Not found: ${req.path}` }));
app.use((err, req, res, _next) => {
  const status = err.status || 500;
  logger.error('HTTP request failed', {
    method: req.method,
    path: req.originalUrl,
    status,
    message: err.message,
    stack: process.env.NODE_ENV === 'production' ? undefined : err.stack
  });
  res.status(status).json({ code: status, message: err.message || 'Internal server error' });
});

export default app;
```

文件：admin/src/config/db.js

> 职责：MySQL 连接池配置和事务封装。
> 模块：后台服务与数据层

```
import mysql from 'mysql2/promise';
import dotenv from 'dotenv';
```

```
3
4  dotenv.config();
5
6  export const pool = mysql.createPool({
7    host: process.env.DB_HOST || 'localhost',
8    port: Number(process.env.DB_PORT || 3306),
9    user: process.env.DB_USER || 'root',
10   password: process.env.DB_PASSWORD || '***',
11   database: process.env.DB_NAME || 'school_chinese_exam_practice',
12   waitForConnections: true,
13   connectionLimit: 10,
14   namedPlaceholders: true
15 });
16
17 export async function tx(work) {
18   const conn = await pool.getConnection();
19   try {
20     await conn.beginTransaction();
21     const result = await work(conn);
22     await conn.commit();
23     return result;
24   } catch (error) {
25     await conn.rollback();
26     throw error;
27   } finally {
28     conn.release();
29   }
30 }
```

文件：admin/src/middleware/auth.js

> 职责：JWT 解析和登录态校验中间件。
> 模块：后台服务与数据层

```
1  import jwt from 'jsonwebtoken';
2  import { HttpError } from '../utils/errors.js';
3
4  export function auth(required = true) {
5    return (req, _res, next) => {
6      const header = req.headers.authorization || '';
7      const token = header.startsWith('Bearer ') ? header.slice(7) : '';
8      if (!token) {
9        if (!required) return next();
10       return next(new HttpError(401, 'Unauthorized'));
11     }
12     try {
13       req.user = jwt.verify(token, process.env.JWT_SECRET || '***');
14       return next();
15     } catch {
16       return next(new HttpError(401, 'Invalid token'));
17     }
18   };
19 }
```

文件：admin/src/middleware/role.js

> 职责：管理员角色授权中间件。
> 模块：后台服务与数据层

```
1  import { HttpError } from '../utils/errors.js';
2
3  export function allow(...roles) {
4    return (req, _res, next) => {
5      if (!req.user || !roles.includes(req.user.role)) {
6        return next(new HttpError(403, 'Forbidden'));
7      }
8      return next();
9    };
10 }
```

文件：admin/src/middleware/requestLogger.js

> 职责：HTTP 请求状态、耗时和用户标识日志中间件。
> 模块：后台服务与数据层

```
import { logger } from '../utils/logger.js';

export function requestLogger(req, res, next) {
  const startedAt = Date.now();
  res.on('finish', () => {
    const durationMs = Date.now() - startedAt;
    logger.info('HTTP request completed', {
      method: req.method,
      path: req.originalUrl,
      status: res.statusCode,
      durationMs,
      userId: req.user?.id || null
    });
  });
  next();
}
```

文件：admin/src/routes/auth.js

> 职责：注册、登录、个人资料查询和资料更新接口。
> 模块：后台服务与数据层

```
import { Router } from 'express';
import bcrypt from 'bcryptjs';
import jwt from 'jsonwebtoken';
import { pool } from '../config/db.js';
import { auth } from '../middleware/auth.js';
import { asyncHandler, ok } from '../utils/response.js';
import { HttpError } from '../utils/errors.js';

const router = Router();
const publicUserFields = 'id, username, name, email, phone, role, student_no, nationality, language, status, created_at';

router.post('/register', asyncHandler(async (req, res) => {
  const { username, password, name, email, phone, student_no, nationality, language = 'zh-CN' } = req.body;
  if (!username || !password) throw new HttpError(400, 'Username and password are required');
  const hash = await bcrypt.hash(password, 10);
  await pool.execute(
    `INSERT INTO users (username, password, name, email, phone, role, student_no, nationality, language, status)
     VALUES (?, ?, ?, ?, ?, 'student', ?, ?, ?, 'active')`,
    [username, hash, name || username, email || null, phone || null, student_no || null, nationality || 'Malaysia', language]
  );
  ok(res, null, 'registered');
}));

router.post('/login', asyncHandler(async (req, res) => {
  const { username, password } = req.body;
  const [rows] = await pool.execute(`SELECT * FROM users WHERE username = ? AND status = 'active'`, [username]);
  const user = rows[0];
  if (!user || !(await bcrypt.compare(password || '', user.password))) {
    throw new HttpError(401, 'Invalid username or password');
  }
  const token = jwt.sign({ id: user.id, username: user.username, role: user.role }, process.env.JWT_SECRET || '***', {
expiresIn: '7d' });
  delete user.password;
  ok(res, { token, user });
}));

router.get('/profile', auth(), asyncHandler(async (req, res) => {
  const [rows] = await pool.execute(`SELECT ${publicUserFields} FROM users WHERE id = ?`, [req.user.id]);
  ok(res, rows[0]);
}));

router.put('/profile', auth(), asyncHandler(async (req, res) => {
  const { name, email, phone, nationality, language } = req.body;
  await pool.execute(
```

```
    'UPDATE users SET name=?, email=?, phone=?, nationality=?, language=? WHERE id=?',
    [name, email, phone, nationality, language, req.user.id]
  );
  ok(res);
}));

export default router;
```

文件：admin/src/routes/learning.js

> 职责：等级、分类、练习列表、试卷详情、答题提交、成绩记录和错题本接口。
> 模块：后台服务与数据层

```
import { Router } from 'express';
import { pool, tx } from '../config/db.js';
import { auth } from '../middleware/auth.js';
import { asyncHandler, ok } from '../utils/response.js';
import { HttpError } from '../utils/errors.js';

const router = Router();

function lang(req) {
  return req.query.lang || req.body.language || 'zh-CN';
}

async function getQuestionsByPaper(paperId, language) {
  const [rows] = await pool.execute(
    `SELECT q.id, q.question_type, q.difficulty, pq.score, qt.title, qt.content, qt.analysis,
            l.code level_code, lt.name level_name, c.code category_code, ct.name category_name
     FROM paper_questions pq
     JOIN questions q ON q.id = pq.question_id
     JOIN question_translations qt ON qt.question_id = q.id AND qt.language_code = ?
     JOIN levels l ON l.id = q.level_id
     JOIN level_translations lt ON lt.level_id = l.id AND lt.language_code = ?
     JOIN question_categories c ON c.id = q.category_id
     JOIN question_category_translations ct ON ct.category_id = c.id AND ct.language_code = ?
     WHERE pq.paper_id = ? AND q.status = 'published'
     ORDER BY pq.sort_order ASC`,
    [language, language, language, paperId]
  );
  if (!rows.length) return [];
  const ids = rows.map((row) => row.id);
  const [options] = await pool.query(
    `SELECT o.id, o.question_id, o.option_key, ot.content
     FROM question_options o
     JOIN question_option_translations ot ON ot.option_id = o.id AND ot.language_code = ?
     WHERE o.question_id IN (?)
     ORDER BY o.question_id, o.sort_order`,
    [language, ids]
  );
  return rows.map((question) => ({
    ...question,
    options: options.filter((option) => option.question_id === question.id).map(({ question_id, ...option }) => option)
  }));
}

router.get('/levels', asyncHandler(async (req, res) => {
  const language = lang(req);
  const [rows] = await pool.execute(
    `SELECT l.id, l.code, lt.name, lt.description
     FROM levels l
     JOIN level_translations lt ON lt.level_id = l.id AND lt.language_code = ?
     WHERE l.status = 'active'
     ORDER BY l.sort_order`,
    [language]
  );
  ok(res, rows);
}));

router.get('/categories', asyncHandler(async (req, res) => {
  const language = lang(req);
  const [rows] = await pool.execute(
```

```
    `SELECT c.id, c.code, ct.name, ct.description
     FROM question_categories c
     JOIN question_category_translations ct ON ct.category_id = c.id AND ct.language_code = ?
     WHERE c.status = 'active'
     ORDER BY c.sort_order`,
    [language]
  );
  ok(res, rows);
}));

router.get('/papers', asyncHandler(async (req, res) => {
  const language = lang(req);
  const [rows] = await pool.execute(
    `SELECT p.id, p.paper_type, p.total_score, p.duration_minutes, pt.title, pt.description, lt.name level_name,
            COUNT(pq.id) question_count
     FROM papers p
     JOIN paper_translations pt ON pt.paper_id = p.id AND pt.language_code = ?
     LEFT JOIN level_translations lt ON lt.level_id = p.level_id AND lt.language_code = ?
     LEFT JOIN paper_questions pq ON pq.paper_id = p.id
     WHERE p.status = 'published'
     GROUP BY p.id, p.paper_type, p.total_score, p.duration_minutes, pt.title, pt.description, lt.name
     ORDER BY p.id DESC`,
    [language, language]
  );
  ok(res, rows);
}));

router.get('/papers/:id', auth(), asyncHandler(async (req, res) => {
  const language = lang(req);
  const paperId = Number(req.params.id);
  const [[paper]] = await pool.execute(
    `SELECT p.id, p.paper_type, p.total_score, p.duration_minutes, pt.title, pt.description
     FROM papers p
     JOIN paper_translations pt ON pt.paper_id = p.id AND pt.language_code = ?
     WHERE p.id = ? AND p.status = 'published'`,
    [language, paperId]
  );
  if (!paper) throw new HttpError(404, 'Paper not found');
  paper.questions = await getQuestionsByPaper(paperId, language);
  ok(res, paper);
}));

router.post('/papers/:id/submit', auth(), asyncHandler(async (req, res) => {
  const paperId = Number(req.params.id);
  const answers = Array.isArray(req.body.answers) ? req.body.answers : [];
  const durationSeconds = Number(req.body.duration_seconds || 0);
  const result = await tx(async (conn) => {
    const [questions] = await conn.execute(
      `SELECT pq.question_id, pq.score, GROUP_CONCAT(o.id ORDER BY o.id) correct_option_ids
       FROM paper_questions pq
       JOIN question_options o ON o.question_id = pq.question_id AND o.is_correct = 1
       WHERE pq.paper_id = ?
       GROUP BY pq.question_id, pq.score`,
      [paperId]
    );
    if (!questions.length) throw new HttpError(404, 'Paper not found');
    const byQuestion = new Map(questions.map((q) => [Number(q.question_id), q]));
    let correctCount = 0;
    let totalScore = 0;
    const answerResults = [];

    for (const item of answers) {
      const questionId = Number(item.question_id);
      const question = byQuestion.get(questionId);
      if (!question) continue;
      const selectedIds = (item.selected_option_ids || []).map(Number).sort((a, b) => a - b);
      const correctIds = String(question.correct_option_ids).split(',').map(Number).sort((a, b) => a - b);
      const isCorrect = JSON.stringify(selectedIds) === JSON.stringify(correctIds);
      const score = isCorrect ? Number(question.score) : 0;
      if (isCorrect) correctCount += 1;
      totalScore += score;
      answerResults.push({ question_id: questionId, selected_option_ids: selectedIds, correct_option_ids: correctIds,
is_correct: isCorrect, score });
    }

```

```
134      const wrongCount = questions.length - correctCount;
135      const [record] = await conn.execute(
136        `INSERT INTO study_records (user_id, paper_id, total_questions, correct_count, wrong_count, total_score,
duration_seconds, started_at, submitted_at)
137         VALUES (?, ?, ?, ?, ?, ?, ?, DATE_SUB(NOW(), INTERVAL ? SECOND), NOW())`,
138        [req.user.id, paperId, questions.length, correctCount, wrongCount, totalScore, durationSeconds, durationSeconds]
139      );
140
141      for (const item of answerResults) {
142        await conn.execute(
143          `INSERT INTO user_answers (user_id, question_id, paper_id, study_record_id, selected_option_ids, is_correct, score)
144           VALUES (?, ?, ?, ?, ?, ?, ?)`,
145          [req.user.id, item.question_id, paperId, record.insertId, item.selected_option_ids.join(','), item.is_correct ? 1 : 0,
item.score]
146        );
147        if (!item.is_correct) {
148          await conn.execute(
149            `INSERT INTO wrong_questions (user_id, question_id, wrong_count, resolved, last_wrong_at)
150             VALUES (?, ?, 1, 0, NOW())
151             ON DUPLICATE KEY UPDATE wrong_count = wrong_count + 1, resolved = 0, last_wrong_at = NOW()`,
152            [req.user.id, item.question_id]
153          );
154        } else {
155          await conn.execute('UPDATE wrong_questions SET resolved = 1 WHERE user_id = ? AND question_id = ?', [req.user.id,
item.question_id]);
156        }
157      }
158
159      return {
160        record_id: record.insertId,
161        total_questions: questions.length,
162        correct_count: correctCount,
163        wrong_count: wrongCount,
164        total_score: totalScore,
165        answers: answerResults
166      };
167    });
168    ok(res, result);
169  }));
170
171  router.get('/records', auth(), asyncHandler(async (req, res) => {
172    const language = lang(req);
173    const [rows] = await pool.execute(
174      `SELECT r.*, pt.title paper_title
175       FROM study_records r
176       LEFT JOIN paper_translations pt ON pt.paper_id = r.paper_id AND pt.language_code = ?
177       WHERE r.user_id = ?
178       ORDER BY r.submitted_at DESC`,
179      [language, req.user.id]
180    );
181    ok(res, rows);
182  }));
183
184  router.get('/wrong-questions', auth(), asyncHandler(async (req, res) => {
185    const language = lang(req);
186    const [rows] = await pool.execute(
187      `SELECT w.id, w.wrong_count, w.resolved, w.last_wrong_at, q.id question_id, qt.title, qt.analysis, ct.name category_name
188       FROM wrong_questions w
189       JOIN questions q ON q.id = w.question_id
190       JOIN question_translations qt ON qt.question_id = q.id AND qt.language_code = ?
191       JOIN question_category_translations ct ON ct.category_id = q.category_id AND ct.language_code = ?
192       WHERE w.user_id = ?
193       ORDER BY w.resolved ASC, w.last_wrong_at DESC`,
194      [language, language, req.user.id]
195    );
196    ok(res, rows);
197  }));
198
199  export default router;
```

文件：admin/src/routes/admin.js

职责：管理台统计、用户列表、题库列表、试卷列表和成绩记录列表接口。
模块：后台服务与数据层

```
import { Router } from 'express';
import { pool } from '../config/db.js';
import { auth } from '../middleware/auth.js';
import { allow } from '../middleware/role.js';
import { asyncHandler, ok } from '../utils/response.js';

const router = Router();
router.use(auth(), allow('admin'));

router.get('/stats', asyncHandler(async (_req, res) => {
  const [[users]] = await pool.execute(`SELECT COUNT(*) count FROM users WHERE role='student'`);
  const [[questions]] = await pool.execute(`SELECT COUNT(*) count FROM questions`);
  const [[papers]] = await pool.execute(`SELECT COUNT(*) count FROM papers`);
  const [[records]] = await pool.execute(`SELECT COUNT(*) count, COALESCE(AVG(total_score),0) avg_score FROM study_records`);
  ok(res, {
    students: users.count,
    questions: questions.count,
    papers: papers.count,
    records: records.count,
    avgScore: Number(records.avg_score).toFixed(1)
  });
}));

router.get('/users', asyncHandler(async (_req, res) => {
  const [rows] = await pool.execute(
    `SELECT id, username, name, email, phone, role, student_no, nationality, language, status, created_at
     FROM users ORDER BY id DESC`
  );
  ok(res, rows);
}));

router.get('/questions', asyncHandler(async (req, res) => {
  const language = req.query.lang || 'zh-CN';
  const [rows] = await pool.execute(
    `SELECT q.id, qt.title, q.question_type, q.difficulty, q.score, q.status, lt.name level_name, ct.name category_name
     FROM questions q
     JOIN question_translations qt ON qt.question_id = q.id AND qt.language_code = ?
     JOIN level_translations lt ON lt.level_id = q.level_id AND lt.language_code = ?
     JOIN question_category_translations ct ON ct.category_id = q.category_id AND ct.language_code = ?
     ORDER BY q.id DESC`,
    [language, language, language]
  );
  ok(res, rows);
}));

router.get('/papers', asyncHandler(async (req, res) => {
  const language = req.query.lang || 'zh-CN';
  const [rows] = await pool.execute(
    `SELECT p.id, pt.title, p.paper_type, p.total_score, p.duration_minutes, p.status, COUNT(pq.id) question_count
     FROM papers p
     JOIN paper_translations pt ON pt.paper_id = p.id AND pt.language_code = ?
     LEFT JOIN paper_questions pq ON pq.paper_id = p.id
     GROUP BY p.id, pt.title, p.paper_type, p.total_score, p.duration_minutes, p.status
     ORDER BY p.id DESC`,
    [language]
  );
  ok(res, rows);
}));

router.get('/records', asyncHandler(async (req, res) => {
  const language = req.query.lang || 'zh-CN';
  const [rows] = await pool.execute(
    `SELECT r.id, u.username, u.name, pt.title paper_title, r.total_questions, r.correct_count, r.wrong_count, r.total_score,
r.submitted_at
     FROM study_records r
     JOIN users u ON u.id = r.user_id
     LEFT JOIN paper_translations pt ON pt.paper_id = r.paper_id AND pt.language_code = ?
     ORDER BY r.submitted_at DESC`,
```

```
    [language]
  );
  ok(res, rows);
}));

export default router;
```

文件：admin/src/utils/errors.js

> 职责：项目运行、构建或说明配置。
> 模块：后台服务与数据层

```
export class HttpError extends Error {
  constructor(status, message) {
    super(message);
    this.status = status;
  }
}
```

文件：admin/src/utils/response.js

> 职责：项目运行、构建或说明配置。
> 模块：后台服务与数据层

```
export function ok(res, data = null, message = 'success') {
  return res.json({ code: 200, message, data });
}

export function asyncHandler(fn) {
  return (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
}
```

文件：admin/src/utils/logger.js

> 职责：日志文件写入、控制台输出和敏感字段脱敏工具。
> 模块：后台服务与数据层

```
import fs from 'node:fs';
import path from 'node:path';
import { fileURLToPath } from 'node:url';

const __dirname = path.dirname(fileURLToPath(import.meta.url));
const root = path.resolve(__dirname, '../..');
const logDir = process.env.LOG_DIR || path.join(root, 'logs');
const logFile = path.join(logDir, 'app.log');

function ensureLogDir() {
  fs.mkdirSync(logDir, { recursive: true });
}

function mask(value) {
  if (value == null) return value;
  const text = typeof value === 'string' ? value : JSON.stringify(value);
  return text
    .replace(/(authorization["']?\s*[:=]\s*["']?Bearer\s+)[^"',\s]+/gi, '$1***')
    .replace(/(password["']?\s*[:=]\s*["']?)[^"',\s]+/gi, '$1***')
    .replace(/(token["']?\s*[:=]\s*["']?)[^"',\s]+/gi, '$1***');
}

function write(level, message, meta) {
  const timestamp = new Date().toISOString();
  const extra = meta === undefined ? '' : ` ${mask(meta)}`;
  const line = `[${timestamp}] [${level}] ${message}${extra}`;
  ensureLogDir();
  fs.appendFileSync(logFile, `${line}\n`, 'utf8');

  if (level === 'ERROR') {
    console.error(line);
```

```
  } else {
    console.log(line);
  }
}

export const logger = {
  info(message, meta) {
    write('INFO', message, meta);
  },
  warn(message, meta) {
    write('WARN', message, meta);
  },
  error(message, meta) {
    write('ERROR', message, meta);
  }
};

export { logFile };
```

文件：admin/src/utils/initDb.js

> 职责：项目运行、构建或说明配置。
> 模块：后台服务与数据层

```
import fs from 'node:fs/promises';
import path from 'node:path';
import { fileURLToPath } from 'node:url';
import mysql from 'mysql2/promise';
import dotenv from 'dotenv';
import { logger } from './logger.js';

dotenv.config();

const __dirname = path.dirname(fileURLToPath(import.meta.url));
const root = path.resolve(__dirname, '../..');

const connection = await mysql.createConnection({
  host: process.env.DB_HOST || 'localhost',
  port: Number(process.env.DB_PORT || 3306),
  user: process.env.DB_USER || 'root',
  password: process.env.DB_PASSWORD || '***',
  multipleStatements: true
});

for (const file of ['schema.sql', 'seed.sql']) {
  const sql = await fs.readFile(path.join(root, 'sql', file), 'utf8');
  await connection.query(sql);
  logger.info(`Executed ${file}`);
}

await connection.end();
logger.info('Database initialized.');
```

文件：admin/src/utils/seedUsers.js

> 职责：项目运行、构建或说明配置。
> 模块：后台服务与数据层

```
import bcrypt from 'bcryptjs';
import { pool } from '../config/db.js';
import { logger } from './logger.js';

const users = [
  ['admin', '***', '系统管理员', 'admin', '***@***', 'Malaysia'],
  ['student', '***', 'Amin', 'student', '***@***', 'Malaysia']
];

for (const [username, password, name, role, email, nationality] of users) {
  const hash = await bcrypt.hash(password, 10);
  await pool.execute(
```

```
    `INSERT INTO users (username, password, name, email, role, nationality, language, status)
     VALUES (?, ?, ?, ?, ?, ?, 'zh-CN', 'active')
     ON DUPLICATE KEY UPDATE password=VALUES(password), name=VALUES(name), role=VALUES(role), email=VALUES(email),
nationality=VALUES(nationality), status='active'`,
    [username, hash, name, email, role, nationality]
  );
}

logger.info('Seed users ready for admin and student accounts');
await pool.end();
```

文件：admin/sql/schema.sql

> 职责：数据库和业务表结构定义脚本。
> 模块：后台服务与数据层

```
CREATE DATABASE IF NOT EXISTS school_chinese_exam_practice DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE school_chinese_exam_practice;

CREATE TABLE IF NOT EXISTS users (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(50) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  name VARCHAR(100),
  email VARCHAR(120),
  phone VARCHAR(30),
  role ENUM('student','admin') NOT NULL DEFAULT 'student',
  student_no VARCHAR(50),
  nationality VARCHAR(80),
  language VARCHAR(20) DEFAULT 'zh-CN',
  status ENUM('active','disabled') NOT NULL DEFAULT 'active',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS levels (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  code VARCHAR(40) NOT NULL UNIQUE,
  sort_order INT NOT NULL DEFAULT 0,
  status ENUM('active','disabled') NOT NULL DEFAULT 'active',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS level_translations (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  level_id BIGINT NOT NULL,
  language_code VARCHAR(20) NOT NULL,
  name VARCHAR(120) NOT NULL,
  description VARCHAR(500),
  UNIQUE KEY uk_level_lang (level_id, language_code),
  CONSTRAINT fk_level_translations_level FOREIGN KEY (level_id) REFERENCES levels(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS question_categories (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  parent_id BIGINT,
  code VARCHAR(60) NOT NULL UNIQUE,
  sort_order INT NOT NULL DEFAULT 0,
  status ENUM('active','disabled') NOT NULL DEFAULT 'active',
  CONSTRAINT fk_categories_parent FOREIGN KEY (parent_id) REFERENCES question_categories(id) ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS question_category_translations (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  category_id BIGINT NOT NULL,
  language_code VARCHAR(20) NOT NULL,
  name VARCHAR(120) NOT NULL,
  description VARCHAR(500),
  UNIQUE KEY uk_category_lang (category_id, language_code),
  CONSTRAINT fk_category_translations_category FOREIGN KEY (category_id) REFERENCES question_categories(id) ON DELETE CASCADE
);

```

```
CREATE TABLE IF NOT EXISTS questions (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  level_id BIGINT NOT NULL,
  category_id BIGINT NOT NULL,
  question_type ENUM('single_choice','multiple_choice','true_false','fill_blank') NOT NULL DEFAULT 'single_choice',
  difficulty ENUM('easy','normal','hard') NOT NULL DEFAULT 'easy',
  score DECIMAL(5,2) NOT NULL DEFAULT 1.00,
  audio_url VARCHAR(500),
  image_url VARCHAR(500),
  status ENUM('published','draft','disabled') NOT NULL DEFAULT 'published',
  created_by BIGINT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_questions_level FOREIGN KEY (level_id) REFERENCES levels(id),
  CONSTRAINT fk_questions_category FOREIGN KEY (category_id) REFERENCES question_categories(id),
  CONSTRAINT fk_questions_creator FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS question_translations (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  question_id BIGINT NOT NULL,
  language_code VARCHAR(20) NOT NULL,
  title VARCHAR(500) NOT NULL,
  content TEXT,
  analysis TEXT,
  UNIQUE KEY uk_question_lang (question_id, language_code),
  CONSTRAINT fk_question_translations_question FOREIGN KEY (question_id) REFERENCES questions(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS question_options (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  question_id BIGINT NOT NULL,
  option_key VARCHAR(10) NOT NULL,
  is_correct TINYINT(1) NOT NULL DEFAULT 0,
  sort_order INT NOT NULL DEFAULT 0,
  UNIQUE KEY uk_question_option_key (question_id, option_key),
  CONSTRAINT fk_options_question FOREIGN KEY (question_id) REFERENCES questions(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS question_option_translations (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  option_id BIGINT NOT NULL,
  language_code VARCHAR(20) NOT NULL,
  content VARCHAR(500) NOT NULL,
  UNIQUE KEY uk_option_lang (option_id, language_code),
  CONSTRAINT fk_option_translations_option FOREIGN KEY (option_id) REFERENCES question_options(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS papers (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  paper_type ENUM('practice','exam','daily') NOT NULL DEFAULT 'practice',
  level_id BIGINT,
  total_score DECIMAL(8,2) NOT NULL DEFAULT 0,
  duration_minutes INT NOT NULL DEFAULT 0,
  status ENUM('published','draft','disabled') NOT NULL DEFAULT 'published',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_papers_level FOREIGN KEY (level_id) REFERENCES levels(id) ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS paper_translations (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  paper_id BIGINT NOT NULL,
  language_code VARCHAR(20) NOT NULL,
  title VARCHAR(200) NOT NULL,
  description VARCHAR(500),
  UNIQUE KEY uk_paper_lang (paper_id, language_code),
  CONSTRAINT fk_paper_translations_paper FOREIGN KEY (paper_id) REFERENCES papers(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS paper_questions (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  paper_id BIGINT NOT NULL,
  question_id BIGINT NOT NULL,
  sort_order INT NOT NULL DEFAULT 0,
```

```
  score DECIMAL(5,2) NOT NULL DEFAULT 1.00,
  UNIQUE KEY uk_paper_question (paper_id, question_id),
  CONSTRAINT fk_paper_questions_paper FOREIGN KEY (paper_id) REFERENCES papers(id) ON DELETE CASCADE,
  CONSTRAINT fk_paper_questions_question FOREIGN KEY (question_id) REFERENCES questions(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS study_records (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  user_id BIGINT NOT NULL,
  paper_id BIGINT,
  total_questions INT NOT NULL DEFAULT 0,
  correct_count INT NOT NULL DEFAULT 0,
  wrong_count INT NOT NULL DEFAULT 0,
  total_score DECIMAL(8,2) NOT NULL DEFAULT 0,
  duration_seconds INT NOT NULL DEFAULT 0,
  started_at DATETIME,
  submitted_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_study_records_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  CONSTRAINT fk_study_records_paper FOREIGN KEY (paper_id) REFERENCES papers(id) ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS user_answers (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  user_id BIGINT NOT NULL,
  question_id BIGINT NOT NULL,
  paper_id BIGINT,
  study_record_id BIGINT,
  selected_option_ids VARCHAR(255),
  answer_text TEXT,
  is_correct TINYINT(1) NOT NULL DEFAULT 0,
  score DECIMAL(5,2) NOT NULL DEFAULT 0,
  answered_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_answers_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  CONSTRAINT fk_answers_question FOREIGN KEY (question_id) REFERENCES questions(id) ON DELETE CASCADE,
  CONSTRAINT fk_answers_paper FOREIGN KEY (paper_id) REFERENCES papers(id) ON DELETE SET NULL,
  CONSTRAINT fk_answers_record FOREIGN KEY (study_record_id) REFERENCES study_records(id) ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS wrong_questions (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  user_id BIGINT NOT NULL,
  question_id BIGINT NOT NULL,
  wrong_count INT NOT NULL DEFAULT 1,
  resolved TINYINT(1) NOT NULL DEFAULT 0,
  last_wrong_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uk_wrong_user_question (user_id, question_id),
  CONSTRAINT fk_wrong_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  CONSTRAINT fk_wrong_question FOREIGN KEY (question_id) REFERENCES questions(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS favorite_questions (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  user_id BIGINT NOT NULL,
  question_id BIGINT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uk_favorite_user_question (user_id, question_id),
  CONSTRAINT fk_favorite_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
  CONSTRAINT fk_favorite_question FOREIGN KEY (question_id) REFERENCES questions(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS i18n_messages (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  module VARCHAR(80) NOT NULL,
  message_key VARCHAR(120) NOT NULL,
  language_code VARCHAR(20) NOT NULL,
  message_value VARCHAR(1000) NOT NULL,
  UNIQUE KEY uk_i18n_message (module, message_key, language_code)
);
```

文件：admin/sql/seed.sql

> 职责：等级、分类、试卷、题目、选项和三语内容初始化脚本。

模块：后台服务与数据层

```sql
USE school_chinese_exam_practice;

INSERT INTO levels (code, sort_order, status) VALUES
('HSK1', 1, 'active'),
('HSK2', 2, 'active'),
('CAMPUS', 3, 'active')
ON DUPLICATE KEY UPDATE sort_order=VALUES(sort_order), status=VALUES(status);

INSERT INTO level_translations (level_id, language_code, name, description)
SELECT l.id, v.language_code, v.name, v.description
FROM levels l
JOIN (
  SELECT 'HSK1' code, 'zh-CN' language_code, 'HSK 1 入门' name, '适合零基础和初级学习者' description UNION ALL
  SELECT 'HSK1', 'en-US', 'HSK 1 Beginner', 'For new and beginner learners' UNION ALL
  SELECT 'HSK1', 'ms-MY', 'HSK 1 Permulaan', 'Untuk pelajar baharu dan tahap asas' UNION ALL
  SELECT 'HSK2', 'zh-CN', 'HSK 2 基础', '掌握常用词汇和基础句型' UNION ALL
  SELECT 'HSK2', 'en-US', 'HSK 2 Elementary', 'Common vocabulary and basic sentence patterns' UNION ALL
  SELECT 'HSK2', 'ms-MY', 'HSK 2 Asas', 'Kosa kata biasa dan pola ayat asas' UNION ALL
  SELECT 'CAMPUS', 'zh-CN', '校园生活汉语', '面向马来西亚留学生的校园场景练习' UNION ALL
  SELECT 'CAMPUS', 'en-US', 'Campus Chinese', 'Campus scenarios for Malaysian international students' UNION ALL
  SELECT 'CAMPUS', 'ms-MY', 'Bahasa Cina Kampus', 'Latihan situasi kampus untuk pelajar Malaysia'
) v ON v.code = l.code
ON DUPLICATE KEY UPDATE name=VALUES(name), description=VALUES(description);

INSERT INTO question_categories (code, sort_order, status) VALUES
('pinyin', 1, 'active'),
('vocabulary', 2, 'active'),
('grammar', 3, 'active'),
('reading', 4, 'active')
ON DUPLICATE KEY UPDATE sort_order=VALUES(sort_order), status=VALUES(status);

INSERT INTO question_category_translations (category_id, language_code, name, description)
SELECT c.id, v.language_code, v.name, v.description
FROM question_categories c
JOIN (
  SELECT 'pinyin' code, 'zh-CN' language_code, '拼音' name, '声母、韵母和声调练习' description UNION ALL
  SELECT 'pinyin', 'en-US', 'Pinyin', 'Initials, finals and tones' UNION ALL
  SELECT 'pinyin', 'ms-MY', 'Pinyin', 'Awalan, akhiran dan nada' UNION ALL
  SELECT 'vocabulary', 'zh-CN', '词汇', '常用汉语词汇' UNION ALL
  SELECT 'vocabulary', 'en-US', 'Vocabulary', 'Common Chinese words' UNION ALL
  SELECT 'vocabulary', 'ms-MY', 'Kosa Kata', 'Perkataan Cina biasa' UNION ALL
  SELECT 'grammar', 'zh-CN', '语法', '基础句型和语序' UNION ALL
  SELECT 'grammar', 'en-US', 'Grammar', 'Basic patterns and word order' UNION ALL
  SELECT 'grammar', 'ms-MY', 'Tatabahasa', 'Pola asas dan susunan kata' UNION ALL
  SELECT 'reading', 'zh-CN', '阅读', '短文理解练习' UNION ALL
  SELECT 'reading', 'en-US', 'Reading', 'Short passage comprehension' UNION ALL
  SELECT 'reading', 'ms-MY', 'Bacaan', 'Pemahaman petikan pendek'
) v ON v.code = c.code
ON DUPLICATE KEY UPDATE name=VALUES(name), description=VALUES(description);

INSERT INTO papers (id, paper_type, level_id, total_score, duration_minutes, status)
SELECT 1, 'practice', id, 5, 15, 'published' FROM levels WHERE code='HSK1'
ON DUPLICATE KEY UPDATE total_score=VALUES(total_score), duration_minutes=VALUES(duration_minutes), status=VALUES(status);

INSERT INTO paper_translations (paper_id, language_code, title, description) VALUES
(1, 'zh-CN', 'HSK1 每日练习', '5 道入门汉语练习题'),
(1, 'en-US', 'HSK1 Daily Practice', 'Five beginner Chinese practice questions'),
(1, 'ms-MY', 'Latihan Harian HSK1', 'Lima soalan latihan Bahasa Cina tahap permulaan')
ON DUPLICATE KEY UPDATE title=VALUES(title), description=VALUES(description);

INSERT INTO questions (id, level_id, category_id, question_type, difficulty, score, status)
SELECT 1, l.id, c.id, 'single_choice', 'easy', 1, 'published' FROM levels l JOIN question_categories c ON c.code='pinyin' WHERE l.code='HSK1'
ON DUPLICATE KEY UPDATE level_id=VALUES(level_id), category_id=VALUES(category_id), status='published';
INSERT INTO questions (id, level_id, category_id, question_type, difficulty, score, status)
SELECT 2, l.id, c.id, 'single_choice', 'easy', 1, 'published' FROM levels l JOIN question_categories c ON c.code='vocabulary' WHERE l.code='HSK1'
ON DUPLICATE KEY UPDATE level_id=VALUES(level_id), category_id=VALUES(category_id), status='published';
INSERT INTO questions (id, level_id, category_id, question_type, difficulty, score, status)
SELECT 3, l.id, c.id, 'single_choice', 'easy', 1, 'published' FROM levels l JOIN question_categories c ON c.code='grammar' WHERE l.code='HSK1'
ON DUPLICATE KEY UPDATE level_id=VALUES(level_id), category_id=VALUES(category_id), status='published';
```

```
INSERT INTO questions (id, level_id, category_id, question_type, difficulty, score, status)
SELECT 4, l.id, c.id, 'single_choice', 'normal', 1, 'published' FROM levels l JOIN question_categories c ON c.code='vocabulary' WHERE l.code='HSK1'
ON DUPLICATE KEY UPDATE level_id=VALUES(level_id), category_id=VALUES(category_id), status='published';
INSERT INTO questions (id, level_id, category_id, question_type, difficulty, score, status)
SELECT 5, l.id, c.id, 'single_choice', 'normal', 1, 'published' FROM levels l JOIN question_categories c ON c.code='reading' WHERE l.code='HSK1'
ON DUPLICATE KEY UPDATE level_id=VALUES(level_id), category_id=VALUES(category_id), status='published';

INSERT INTO question_translations (question_id, language_code, title, content, analysis) VALUES
(1, 'zh-CN', '“你”的拼音是什么？', '请选择正确答案。', '“你”的拼音是 ni，第三声。'),
(1, 'en-US', 'What is the pinyin for “你”?', 'Choose the correct answer.', 'The pinyin for 你 is ni with the third tone.'),
(1, 'ms-MY', 'Apakah pinyin bagi “你”?', 'Pilih jawapan yang betul.', 'Pinyin untuk 你 ialah ni dengan nada ketiga.'),
(2, 'zh-CN', '“谢谢”是什么意思？', '请选择最合适的英文意思。', '“谢谢”表示感谢。'),
(2, 'en-US', 'What does “谢谢” mean?', 'Choose the best English meaning.', '谢谢 means thank you.'),
(2, 'ms-MY', 'Apakah maksud “谢谢”?', 'Pilih maksud Bahasa Inggeris yang paling sesuai.', '谢谢 bermaksud terima kasih.'),
(3, 'zh-CN', '选择正确的句子。', '哪一句的语序正确？', '汉语常用语序是主语 + 谓语 + 宾语。'),
(3, 'en-US', 'Choose the correct sentence.', 'Which sentence has the correct word order?', 'Basic Chinese word order is subject + verb + object.'),
(3, 'ms-MY', 'Pilih ayat yang betul.', 'Ayat manakah mempunyai susunan kata yang betul?', 'Susunan kata asas Bahasa Cina ialah subjek + kata kerja + objek.'),
(4, 'zh-CN', '“学校”的意思是？', '请选择正确答案。', '学校是学习的地方。'),
(4, 'en-US', 'What does “学校” mean?', 'Choose the correct answer.', '学校 means school.'),
(4, 'ms-MY', 'Apakah maksud “学校”?', 'Pilih jawapan yang betul.', '学校 bermaksud sekolah.'),
(5, 'zh-CN', '阅读：我叫阿明。我是马来西亚学生。我学习汉语。阿明来自哪里？', '请选择正确答案。', '短文说“我是马来西亚学生”。'),
(5, 'en-US', 'Reading: My name is Amin. I am a Malaysian student. I study Chinese. Where is Amin from?', 'Choose the correct answer.', 'The passage says he is a Malaysian student.'),
(5, 'ms-MY', 'Bacaan: Nama saya Amin. Saya pelajar Malaysia. Saya belajar Bahasa Cina. Amin berasal dari mana?', 'Pilih jawapan yang betul.', 'Petikan menyatakan dia pelajar Malaysia.')
ON DUPLICATE KEY UPDATE title=VALUES(title), content=VALUES(content), analysis=VALUES(analysis);

INSERT INTO question_options (id, question_id, option_key, is_correct, sort_order) VALUES
(1, 1, 'A', 1, 1), (2, 1, 'B', 0, 2), (3, 1, 'C', 0, 3), (4, 1, 'D', 0, 4),
(5, 2, 'A', 0, 1), (6, 2, 'B', 1, 2), (7, 2, 'C', 0, 3), (8, 2, 'D', 0, 4),
(9, 3, 'A', 1, 1), (10, 3, 'B', 0, 2), (11, 3, 'C', 0, 3), (12, 3, 'D', 0, 4),
(13, 4, 'A', 0, 1), (14, 4, 'B', 1, 2), (15, 4, 'C', 0, 3), (16, 4, 'D', 0, 4),
(17, 5, 'A', 0, 1), (18, 5, 'B', 1, 2), (19, 5, 'C', 0, 3), (20, 5, 'D', 0, 4)
ON DUPLICATE KEY UPDATE is_correct=VALUES(is_correct), sort_order=VALUES(sort_order);

INSERT INTO question_option_translations (option_id, language_code, content) VALUES
(1, 'zh-CN', 'nǐ'), (1, 'en-US', 'nǐ'), (1, 'ms-MY', 'nǐ'),
(2, 'zh-CN', 'wǒ'), (2, 'en-US', 'wǒ'), (2, 'ms-MY', 'wǒ'),
(3, 'zh-CN', 'tā'), (3, 'en-US', 'tā'), (3, 'ms-MY', 'tā'),
(4, 'zh-CN', 'hǎo'), (4, 'en-US', 'hǎo'), (4, 'ms-MY', 'hǎo'),
(5, 'zh-CN', 'Hello'), (5, 'en-US', 'Hello'), (5, 'ms-MY', 'Hello'),
(6, 'zh-CN', 'Thank you'), (6, 'en-US', 'Thank you'), (6, 'ms-MY', 'Thank you'),
(7, 'zh-CN', 'Goodbye'), (7, 'en-US', 'Goodbye'), (7, 'ms-MY', 'Goodbye'),
(8, 'zh-CN', 'Teacher'), (8, 'en-US', 'Teacher'), (8, 'ms-MY', 'Teacher'),
(9, 'zh-CN', '我学习汉语。'), (9, 'en-US', '我学习汉语。'), (9, 'ms-MY', '我学习汉语。'),
(10, 'zh-CN', '学习我汉语。'), (10, 'en-US', '学习我汉语。'), (10, 'ms-MY', '学习我汉语。'),
(11, 'zh-CN', '汉语我学习。'), (11, 'en-US', '汉语我学习。'), (11, 'ms-MY', '汉语我学习。'),
(12, 'zh-CN', '我汉语学习。'), (12, 'en-US', '我汉语学习。'), (12, 'ms-MY', '我汉语学习。'),
(13, 'zh-CN', 'Hospital'), (13, 'en-US', 'Hospital'), (13, 'ms-MY', 'Hospital'),
(14, 'zh-CN', 'School'), (14, 'en-US', 'School'), (14, 'ms-MY', 'School'),
(15, 'zh-CN', 'Library'), (15, 'en-US', 'Library'), (15, 'ms-MY', 'Library'),
(16, 'zh-CN', 'Restaurant'), (16, 'en-US', 'Restaurant'), (16, 'ms-MY', 'Restaurant'),
(17, 'zh-CN', '中国'), (17, 'en-US', 'China'), (17, 'ms-MY', 'China'),
(18, 'zh-CN', '马来西亚'), (18, 'en-US', 'Malaysia'), (18, 'ms-MY', 'Malaysia'),
(19, 'zh-CN', '日本'), (19, 'en-US', 'Japan'), (19, 'ms-MY', 'Jepun'),
(20, 'zh-CN', '英国'), (20, 'en-US', 'United Kingdom'), (20, 'ms-MY', 'United Kingdom')
ON DUPLICATE KEY UPDATE content=VALUES(content);

INSERT INTO paper_questions (paper_id, question_id, sort_order, score) VALUES
(1, 1, 1, 1), (1, 2, 2, 1), (1, 3, 3, 1), (1, 4, 4, 1), (1, 5, 5, 1)
ON DUPLICATE KEY UPDATE sort_order=VALUES(sort_order), score=VALUES(score);
```

文件：fronter/src/main.js

> 职责：项目运行、构建或说明配置。
> 模块：前台 H5 页面层

```
import { createApp } from 'vue';
import App from './App.vue';
import router from './router/index.js';
import './assets/style.css';

createApp(App).use(router).mount('#app');
```

文件：fronter/src/App.vue

> 职责：Vue 前台或管理端页面组件。
> 模块：前台 H5 页面层

```
<template>
  <div class="app-shell">
    <header class="topbar">
      <router-link class="brand" to="/">{{ t('app') }}</router-link>
      <nav class="toolbar">
        <router-link to="/">{{ t('home') }}</router-link>
        <router-link v-if="auth.user" to="/practice">{{ t('practice') }}</router-link>
        <router-link v-if="auth.user" to="/records">{{ t('records') }}</router-link>
        <router-link v-if="auth.user" to="/wrong-book">{{ t('wrongBook') }}</router-link>
        <router-link v-if="auth.user" to="/profile">{{ t('profile') }}</router-link>
        <router-link v-if="auth.user?.role === 'admin'" to="/admin">{{ t('dashboard') }}</router-link>
        <router-link v-if="!auth.user" to="/login">{{ t('login') }}</router-link>
        <button v-if="auth.user" @click="logout">{{ t('logout') }}</button>
        <select :value="state.lang" @change="setLang($event.target.value)">
          <option value="zh-CN">中文</option>
          <option value="en-US">English</option>
          <option value="ms-MY">Bahasa Melayu</option>
        </select>
      </nav>
    </header>
    <main class="content">
      <router-view />
    </main>
  </div>
</template>

<script setup>
import { useRouter } from 'vue-router';
import { authStore as auth } from './stores/auth.js';
import { state, t, setLang } from './i18n/index.js';

const router = useRouter();
function logout() {
  auth.logout();
  router.push('/login');
}
</script>
```

文件：fronter/src/router/index.js

> 职责：前台路由和权限守卫定义。
> 模块：前台 H5 页面层

```
import { createRouter, createWebHistory } from 'vue-router';
import { authStore } from '../stores/auth.js';
import Login from '../views/Login.vue';
import Register from '../views/Register.vue';
import Home from '../views/Home.vue';
import PracticeList from '../views/PracticeList.vue';
import PracticeDetail from '../views/PracticeDetail.vue';
import Records from '../views/Records.vue';
import WrongBook from '../views/WrongBook.vue';
import Profile from '../views/Profile.vue';
import Dashboard from '../views/Dashboard.vue';
import AdminTable from '../views/AdminTable.vue';

const routes = [
  { path: '/', component: Home },
```

```
  { path: '/login', component: Login },
  { path: '/register', component: Register },
  { path: '/practice', component: PracticeList, meta: { auth: true } },
  { path: '/practice/:id', component: PracticeDetail, meta: { auth: true } },
  { path: '/records', component: Records, meta: { auth: true } },
  { path: '/wrong-book', component: WrongBook, meta: { auth: true } },
  { path: '/profile', component: Profile, meta: { auth: true } },
  { path: '/admin', component: Dashboard, meta: { auth: true, admin: true } },
  { path: '/admin/users', component: AdminTable, props: { type: 'users' }, meta: { auth: true, admin: true } },
  { path: '/admin/questions', component: AdminTable, props: { type: 'questions' }, meta: { auth: true, admin: true } },
  { path: '/admin/papers', component: AdminTable, props: { type: 'papers' }, meta: { auth: true, admin: true } },
  { path: '/admin/records', component: AdminTable, props: { type: 'records' }, meta: { auth: true, admin: true } }
];

const router = createRouter({ history: createWebHistory(), routes });

router.beforeEach((to) => {
  if (to.meta.auth && !authStore.isAuthed) return '/login';
  if (to.meta.admin && authStore.user?.role !== 'admin') return '/';
  return true;
});

export default router;
```

文件：fronter/src/api/client.js

> 职责：HTTP 请求、JSON 载荷和 Authorization 请求头封装。
> 模块：前台 H5 页面层

```
const API_BASE = import.meta.env.VITE_API_BASE || 'http://***/api';

export function token() {
  return localStorage.getItem('token') || '';
}

export async function request(path, options = {}) {
  const headers = { 'Content-Type': 'application/json', ...(options.headers || {}) };
  if (token()) headers.Authorization = `Bearer ${token()}`;
  const response = await fetch(`${API_BASE}${path}`, {
    ...options,
    headers,
    body: options.body ? JSON.stringify(options.body) : undefined
  });
  const payload = await response.json().catch(() => ({ message: 'Network error' }));
  if (!response.ok || payload.code >= 400) throw new Error(payload.message || 'Request failed');
  return payload.data;
}
```

文件：fronter/src/stores/auth.js

> 职责：项目运行、构建或说明配置。
> 模块：前台 H5 页面层

```
import { reactive } from 'vue';
import { request } from '../api/client.js';

export const authStore = reactive({
  user: JSON.parse(localStorage.getItem('user') || 'null'),
  get isAuthed() {
    return Boolean(localStorage.getItem('token'));
  },
  async login(form) {
    const data = await request('/auth/login', { method: 'POST', body: form });
    localStorage.setItem('token', data.token);
    localStorage.setItem('user', JSON.stringify(data.user));
    this.user = data.user;
  },
  logout() {
    localStorage.removeItem('token');
    localStorage.removeItem('user');
```

```
    this.user = null;
  }
});
```

文件：fronter/src/i18n/index.js

> 职责：三语界面文案和语言状态维护。
> 模块：前台 H5 页面层

```
import { reactive } from 'vue';

export const state = reactive({
  lang: localStorage.getItem('lang') || 'zh-CN'
});

const messages = {
  'zh-CN': {
    app: '马来西亚留学生汉语练习平台',
    login: '登录',
    register: '注册',
    logout: '退出',
    home: '首页',
    practice: '练习',
    records: '成绩',
    wrongBook: '错题本',
    profile: '个人中心',
    dashboard: '管理台',
    users: '学生管理',
    questions: '题库',
    papers: '练习/试卷',
    adminRecords: '成绩记录',
    username: '用户名',
    password: '密码',
    name: '姓名',
    email: '邮箱',
    phone: '电话',
    studentNo: '学号',
    nationality: '国籍',
    language: '语言',
    start: '开始练习',
    submit: '提交',
    next: '下一题',
    result: '练习结果',
    correct: '正确',
    wrong: '错误',
    score: '得分',
    questionCount: '题数',
    duration: '时长',
    level: '等级',
    category: '分类',
    difficulty: '难度',
    status: '状态',
    role: '角色',
    title: '标题',
    createdAt: '创建时间',
    submittedAt: '提交时间',
    empty: '暂无数据',
    chooseAnswer: '请选择答案',
    analysis: '解析',
    totalStudents: '学生数',
    totalQuestions: '题目数',
    totalPapers: '练习数',
    totalRecords: '答题记录',
    avgScore: '平均分'
  },
  'en-US': {
    app: 'Chinese Practice Platform for Malaysian Students',
    login: 'Login',
    register: 'Register',
    logout: 'Logout',
    home: 'Home',
    practice: 'Practice',
```

```
    records: 'Scores',
    wrongBook: 'Wrong Book',
    profile: 'Profile',
    dashboard: 'Admin',
    users: 'Students',
    questions: 'Question Bank',
    papers: 'Practices',
    adminRecords: 'Score Records',
    username: 'Username',
    password: 'Password',
    name: 'Name',
    email: 'Email',
    phone: 'Phone',
    studentNo: 'Student No.',
    nationality: 'Nationality',
    language: 'Language',
    start: 'Start',
    submit: 'Submit',
    next: 'Next',
    result: 'Result',
    correct: 'Correct',
    wrong: 'Wrong',
    score: 'Score',
    questionCount: 'Questions',
    duration: 'Duration',
    level: 'Level',
    category: 'Category',
    difficulty: 'Difficulty',
    status: 'Status',
    role: 'Role',
    title: 'Title',
    createdAt: 'Created',
    submittedAt: 'Submitted',
    empty: 'No data',
    chooseAnswer: 'Please choose an answer',
    analysis: 'Analysis',
    totalStudents: 'Students',
    totalQuestions: 'Questions',
    totalPapers: 'Practices',
    totalRecords: 'Records',
    avgScore: 'Avg Score'
  },
  'ms-MY': {
    app: 'Platform Latihan Bahasa Cina untuk Pelajar Malaysia',
    login: 'Log Masuk',
    register: 'Daftar',
    logout: 'Log Keluar',
    home: 'Laman Utama',
    practice: 'Latihan',
    records: 'Markah',
    wrongBook: 'Buku Salah',
    profile: 'Profil',
    dashboard: 'Admin',
    users: 'Pelajar',
    questions: 'Bank Soalan',
    papers: 'Latihan',
    adminRecords: 'Rekod Markah',
    username: 'Nama Pengguna',
    password: 'Kata Laluan',
    name: 'Nama',
    email: 'E-mel',
    phone: 'Telefon',
    studentNo: 'No. Pelajar',
    nationality: 'Kewarganegaraan',
    language: 'Bahasa',
    start: 'Mula',
    submit: 'Hantar',
    next: 'Seterusnya',
    result: 'Keputusan',
    correct: 'Betul',
    wrong: 'Salah',
    score: 'Markah',
    questionCount: 'Soalan',
    duration: 'Tempoh',
    level: 'Tahap',
```

```
    category: 'Kategori',
    difficulty: 'Kesukaran',
    status: 'Status',
    role: 'Peranan',
    title: 'Tajuk',
    createdAt: 'Dicipta',
    submittedAt: 'Dihantar',
    empty: 'Tiada data',
    chooseAnswer: 'Sila pilih jawapan',
    analysis: 'Analisis',
    totalStudents: 'Pelajar',
    totalQuestions: 'Soalan',
    totalPapers: 'Latihan',
    totalRecords: 'Rekod',
    avgScore: 'Purata'
  }
};

export function t(key) {
  return messages[state.lang]?.[key] || key;
}

export function setLang(lang) {
  state.lang = lang;
  localStorage.setItem('lang', lang);
}
```

文件：fronter/src/views/Home.vue

> 职责：Vue 前台或管理端页面组件。
> 模块：前台 H5 页面层

```
<template>
  <section class="hero">
    <div>
      <h1>{{ t('app') }}</h1>
      <p class="muted">HSK, campus Chinese, trilingual question bank and practice records.</p>
    </div>
    <div class="grid">
      <router-link class="card" to="/practice">
        <h3>{{ t('practice') }}</h3>
        <p class="muted">HSK1 / HSK2 / Campus Chinese</p>
      </router-link>
      <router-link class="card" to="/wrong-book">
        <h3>{{ t('wrongBook') }}</h3>
        <p class="muted">Review mistakes and explanations</p>
      </router-link>
      <router-link class="card" to="/records">
        <h3>{{ t('records') }}</h3>
        <p class="muted">Track practice scores</p>
      </router-link>
      <router-link v-if="auth.user?.role === 'admin'" class="card" to="/admin">
        <h3>{{ t('dashboard') }}</h3>
        <p class="muted">Question bank and student records</p>
      </router-link>
    </div>
  </section>
</template>

<script setup>
import { t } from '../i18n/index.js';
import { authStore as auth } from '../stores/auth.js';
</script>
```

文件：fronter/src/views/Login.vue

> 职责：Vue 前台或管理端页面组件。
> 模块：前台 H5 页面层

```vue
<template>
  <section>
    <h1>{{ t('login') }}</h1>
    <form class="form" @submit.prevent="submit">
      <input v-model="form.username" :placeholder="t('username')" required />
      <input v-model="form.password" :placeholder="t('password')" type="password" required />
      <button class="btn">{{ t('login') }}</button>
      <router-link to="/register">{{ t('register') }}</router-link>
      <p class="muted">{{ message }}</p>
    </form>
  </section>
</template>

<script setup>
import { reactive, ref } from 'vue';
import { useRouter } from 'vue-router';
import { authStore } from '../stores/auth.js';
import { t } from '../i18n/index.js';

const router = useRouter();
const message = ref('');
const form = reactive({ username: 'student', password: '***' });

async function submit() {
  try {
    await authStore.login(form);
    router.push('/');
  } catch (e) {
    message.value = e.message;
  }
}
</script>
```

文件：fronter/src/views/Register.vue

> 职责：Vue 前台或管理端页面组件。
> 模块：前台 H5 页面层

```vue
<template>
  <section>
    <h1>{{ t('register') }}</h1>
    <form class="form" @submit.prevent="submit">
      <input v-model="form.username" :placeholder="t('username')" required />
      <input v-model="form.password" :placeholder="t('password')" type="password" required />
      <input v-model="form.name" :placeholder="t('name')" />
      <input v-model="form.email" :placeholder="t('email')" />
      <input v-model="form.phone" :placeholder="t('phone')" />
      <input v-model="form.student_no" :placeholder="t('studentNo')" />
      <input v-model="form.nationality" :placeholder="t('nationality')" />
      <button class="btn">{{ t('register') }}</button>
      <p class="muted">{{ message }}</p>
    </form>
  </section>
</template>

<script setup>
import { reactive, ref } from 'vue';
import { request } from '../api/client.js';
import { t, state } from '../i18n/index.js';

const message = ref('');
const form = reactive({ username: '', password: '', name: '', email: '', phone: '', student_no: '', nationality: 'Malaysia',
language: state.lang });

async function submit() {
  try {
    await request('/auth/register', { method: 'POST', body: form });
    message.value = 'registered';
  } catch (e) {
    message.value = e.message;
  }
```

```
}
</script>
```

文件：fronter/src/views/PracticeList.vue

> 职责：Vue 前台或管理端页面组件。
> 模块：前台 H5 页面层

```
<template>
  <section>
    <h1>{{ t('practice') }}</h1>
    <div class="grid">
      <router-link v-for="paper in papers" :key="paper.id" class="card" :to="`/practice/${paper.id}`">
        <h3>{{ paper.title }}</h3>
        <p class="muted">{{ paper.description }}</p>
        <div class="row">
          <span class="pill">{{ paper.level_name }}</span>
          <span class="pill">{{ t('questionCount') }}: {{ paper.question_count }}</span>
          <span class="pill">{{ t('score') }}: {{ paper.total_score }}</span>
        </div>
      </router-link>
    </div>
    <p v-if="!papers.length" class="muted">{{ t('empty') }}</p>
  </section>
</template>

<script setup>
import { onMounted, ref, watch } from 'vue';
import { request } from '../api/client.js';
import { state, t } from '../i18n/index.js';

const papers = ref([]);
async function load() {
  papers.value = await request(`/learning/papers?lang=${state.lang}`);
}
onMounted(load);
watch(() => state.lang, load);
</script>
```

文件：fronter/src/views/PracticeDetail.vue

> 职责：Vue 前台或管理端页面组件。
> 模块：前台 H5 页面层

```
<template>
  <section v-if="paper">
    <div class="row">
      <h1>{{ paper.title }}</h1>
      <span class="pill">{{ index + 1 }} / {{ paper.questions.length }}</span>
    </div>
    <div v-if="!result" class="split">
      <div class="card">
        <p class="muted">{{ current.category_name }} · {{ current.difficulty }}</p>
        <h3>{{ current.title }}</h3>
        <p>{{ current.content }}</p>
        <button
          v-for="option in current.options"
          :key="option.id"
          class="option"
          :class="{ active: selectedIds.includes(option.id) }"
          @click="select(option.id)"
        >
          {{ option.option_key }}. {{ option.content }}
        </button>
        <div class="row">
          <button class="btn ghost" :disabled="index === 0" @click="index -= 1">‹</button>
          <button v-if="index < paper.questions.length - 1" class="btn" @click="index += 1">{{ t('next') }}</button>
          <button v-else class="btn" @click="submit">{{ t('submit') }}</button>
        </div>
        <p class="muted">{{ message }}</p>
```

```
      </div>
      <aside class="card">
        <h4>{{ t('questionCount') }}</h4>
        <div class="row">
          <button v-for="(q, i) in paper.questions" :key="q.id" class="btn ghost" @click="index = i">
            {{ i + 1 }}
          </button>
        </div>
      </aside>
    </div>
    <div v-else class="card">
      <h2>{{ t('result') }}</h2>
      <div class="grid">
        <div><strong>{{ result.correct_count }}</strong><p>{{ t('correct') }}</p></div>
        <div><strong>{{ result.wrong_count }}</strong><p>{{ t('wrong') }}</p></div>
        <div><strong>{{ result.total_score }}</strong><p>{{ t('score') }}</p></div>
      </div>
      <div v-for="question in paper.questions" :key="question.id" class="card">
        <h4>{{ question.title }}</h4>
        <p class="muted">{{ question.analysis }}</p>
      </div>
    </div>
  </section>
</template>

<script setup>
import { computed, onMounted, reactive, ref, watch } from 'vue';
import { useRoute } from 'vue-router';
import { request } from '../api/client.js';
import { state, t } from '../i18n/index.js';

const route = useRoute();
const paper = ref(null);
const index = ref(0);
const result = ref(null);
const message = ref('');
const answers = reactive({});
const startedAt = Date.now();
const current = computed(() => paper.value?.questions[index.value] || {});
const selectedIds = computed(() => answers[current.value.id] || []);

async function load() {
  paper.value = await request(`/learning/papers/${route.params.id}?lang=${state.lang}`);
  index.value = 0;
  result.value = null;
}
function select(optionId) {
  answers[current.value.id] = [optionId];
  message.value = '';
}
async function submit() {
  const payload = paper.value.questions.map((question) => ({ question_id: question.id, selected_option_ids:
answers[question.id] || [] }));
  if (payload.some((item) => item.selected_option_ids.length === 0)) {
    message.value = t('chooseAnswer');
    return;
  }
  result.value = await request(`/learning/papers/${paper.value.id}/submit`, {
    method: 'POST',
    body: { answers: payload, duration_seconds: Math.round((Date.now() - startedAt) / 1000), language: state.lang }
  });
}
onMounted(load);
watch(() => state.lang, load);
</script>
```

文件：fronter/src/views/Records.vue

> 职责：Vue 前台或管理端页面组件。
> 模块：前台 H5 页面层

```vue
<template>
  <section>
    <h1>{{ t('records') }}</h1>
    <div class="card" style="overflow:auto">
      <table>
        <thead>
          <tr>
            <th>{{ t('title') }}</th>
            <th>{{ t('questionCount') }}</th>
            <th>{{ t('correct') }}</th>
            <th>{{ t('wrong') }}</th>
            <th>{{ t('score') }}</th>
            <th>{{ t('submittedAt') }}</th>
          </tr>
        </thead>
        <tbody>
          <tr v-for="row in rows" :key="row.id">
            <td>{{ row.paper_title }}</td>
            <td>{{ row.total_questions }}</td>
            <td>{{ row.correct_count }}</td>
            <td>{{ row.wrong_count }}</td>
            <td>{{ row.total_score }}</td>
            <td>{{ row.submitted_at }}</td>
          </tr>
        </tbody>
      </table>
      <p v-if="!rows.length" class="muted">{{ t('empty') }}</p>
    </div>
  </section>
</template>

<script setup>
import { onMounted, ref, watch } from 'vue';
import { request } from '../api/client.js';
import { state, t } from '../i18n/index.js';

const rows = ref([]);
async function load() {
  rows.value = await request(`/learning/records?lang=${state.lang}`);
}
onMounted(load);
watch(() => state.lang, load);
</script>
```

文件：fronter/src/views/WrongBook.vue

> 职责：Vue 前台或管理端页面组件。
> 模块：前台 H5 页面层

```vue
<template>
  <section>
    <h1>{{ t('wrongBook') }}</h1>
    <div class="grid">
      <article v-for="row in rows" :key="row.id" class="card">
        <div class="row">
          <span class="pill">{{ row.category_name }}</span>
          <span class="pill">{{ t('wrong') }}: {{ row.wrong_count }}</span>
        </div>
        <h3>{{ row.title }}</h3>
        <p class="muted">{{ t('analysis') }}: {{ row.analysis }}</p>
      </article>
    </div>
    <p v-if="!rows.length" class="muted">{{ t('empty') }}</p>
  </section>
</template>

<script setup>
import { onMounted, ref, watch } from 'vue';
import { request } from '../api/client.js';
import { state, t } from '../i18n/index.js';

const rows = ref([]);
```

```
async function load() {
  rows.value = await request(`/learning/wrong-questions?lang=${state.lang}`);
}
onMounted(load);
watch(() => state.lang, load);
</script>
```

文件：fronter/src/views/Profile.vue

> 职责：Vue 前台或管理端页面组件。
> 模块：前台 H5 页面层

```
<template>
  <section>
    <h1>{{ t('profile') }}</h1>
    <form class="form" @submit.prevent="save">
      <input v-model="form.name" :placeholder="t('name')" />
      <input v-model="form.email" :placeholder="t('email')" />
      <input v-model="form.phone" :placeholder="t('phone')" />
      <input v-model="form.nationality" :placeholder="t('nationality')" />
      <select v-model="form.language">
        <option value="zh-CN">中文</option>
        <option value="en-US">English</option>
        <option value="ms-MY">Bahasa Melayu</option>
      </select>
      <button class="btn">{{ t('submit') }}</button>
      <p class="muted">{{ message }}</p>
    </form>
  </section>
</template>

<script setup>
import { onMounted, reactive, ref } from 'vue';
import { request } from '../api/client.js';
import { setLang, t } from '../i18n/index.js';

const message = ref('');
const form = reactive({ name: '', email: '', phone: '', nationality: '', language: 'zh-CN' });

onMounted(async () => {
  Object.assign(form, await request('/auth/profile'));
});

async function save() {
  await request('/auth/profile', { method: 'PUT', body: form });
  setLang(form.language);
  message.value = 'saved';
}
</script>
```

文件：fronter/src/views/Dashboard.vue

> 职责：Vue 前台或管理端页面组件。
> 模块：前台 H5 页面层

```
<template>
  <section>
    <h1>{{ t('dashboard') }}</h1>
    <div class="grid">
      <div class="card"><h3>{{ stats.students }}</h3><p>{{ t('totalStudents') }}</p></div>
      <div class="card"><h3>{{ stats.questions }}</h3><p>{{ t('totalQuestions') }}</p></div>
      <div class="card"><h3>{{ stats.papers }}</h3><p>{{ t('totalPapers') }}</p></div>
      <div class="card"><h3>{{ stats.avgScore }}</h3><p>{{ t('avgScore') }}</p></div>
    </div>
    <div class="grid" style="margin-top:12px">
      <router-link class="card" to="/admin/users">{{ t('users') }}</router-link>
      <router-link class="card" to="/admin/questions">{{ t('questions') }}</router-link>
      <router-link class="card" to="/admin/papers">{{ t('papers') }}</router-link>
      <router-link class="card" to="/admin/records">{{ t('adminRecords') }}</router-link>
    </div>
```

```
  </section>
</template>

<script setup>
import { onMounted, reactive } from 'vue';
import { request } from '../api/client.js';
import { t } from '../i18n/index.js';

const stats = reactive({ students: 0, questions: 0, papers: 0, records: 0, avgScore: 0 });
onMounted(async () => Object.assign(stats, await request('/admin/stats')));
</script>
```

文件：fronter/src/views/AdminTable.vue

> 职责：Vue 前台或管理端页面组件。
> 模块：前台 H5 页面层

```
<template>
  <section>
    <h1>{{ title }}</h1>
    <div class="card" style="overflow:auto">
      <table>
        <thead>
          <tr>
            <th v-for="header in headers" :key="header">{{ label(header) }}</th>
          </tr>
        </thead>
        <tbody>
          <tr v-for="row in rows" :key="row.id">
            <td v-for="header in headers" :key="header">{{ format(row[header]) }}</td>
          </tr>
        </tbody>
      </table>
      <p v-if="!rows.length" class="muted">{{ t('empty') }}</p>
    </div>
  </section>
</template>

<script setup>
import { computed, onMounted, ref, watch } from 'vue';
import { request } from '../api/client.js';
import { state, t } from '../i18n/index.js';

const props = defineProps({ type: { type: String, required: true } });
const rows = ref([]);
const title = computed(() => t(props.type === 'records' ? 'adminRecords' : props.type));
const headerMap = {
  users: ['id', 'username', 'name', 'role', 'student_no', 'nationality', 'language', 'status'],
  questions: ['id', 'title', 'level_name', 'category_name', 'question_type', 'difficulty', 'score', 'status'],
  papers: ['id', 'title', 'paper_type', 'question_count', 'total_score', 'duration_minutes', 'status'],
  records: ['id', 'username', 'name', 'paper_title', 'total_questions', 'correct_count', 'wrong_count', 'total_score',
'submitted_at']
};
const labelMap = {
  id: 'ID',
  username: t('username'),
  name: t('name'),
  role: t('role'),
  student_no: t('studentNo'),
  nationality: t('nationality'),
  language: t('language'),
  status: t('status'),
  title: t('title'),
  level_name: t('level'),
  category_name: t('category'),
  question_type: 'Type',
  difficulty: t('difficulty'),
  score: t('score'),
  paper_type: 'Type',
  question_count: t('questionCount'),
  total_score: t('score'),
  duration_minutes: t('duration'),
```

```
  paper_title: t('title'),
  total_questions: t('questionCount'),
  correct_count: t('correct'),
  wrong_count: t('wrong'),
  submitted_at: t('submittedAt')
};
const headers = computed(() => headerMap[props.type] || ['id']);

function label(key) {
  return labelMap[key] || key;
}
function format(value) {
  if (value == null) return '';
  if (typeof value === 'string' && value.length > 80) return `${value.slice(0, 80)}...`;
  return value;
}
async function load() {
  rows.value = await request(`/admin/${props.type}?lang=${state.lang}`);
}
onMounted(load);
watch(() => [props.type, state.lang], load);
</script>
```

文件：fronter/src/assets/style.css

> 职责：前台页面通用样式。
> 模块：前台 H5 页面层

```
* { box-sizing: border-box; }
body { margin: 0; font-family: Arial, "Microsoft YaHei", sans-serif; background: #f4f6f8; color: #172033; }
a { color: inherit; text-decoration: none; }
button, input, select, textarea { font: inherit; }
.app-shell { max-width: 1080px; min-height: 100vh; margin: 0 auto; background: #fff; }
.topbar { position: sticky; top: 0; z-index: 5; display: flex; gap: 12px; align-items: center; justify-content: space-between; padding: 12px 14px; background: #14304d; color: #fff; }
.brand { font-weight: 700; line-height: 1.3; }
.toolbar { display: flex; gap: 8px; align-items: center; flex-wrap: wrap; }
.toolbar a, .toolbar button, .toolbar select { border: 0; border-radius: 6px; padding: 8px 10px; background: rgba(255,255,255,.12); color: #fff; cursor: pointer; }
.toolbar select option { color: #172033; }
.content { padding: 16px; }
.hero { display: grid; gap: 14px; padding: 20px 0; }
.grid { display: grid; gap: 12px; grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); }
.card { border: 1px solid #e4e8ef; border-radius: 8px; padding: 14px; background: #fff; box-shadow: 0 1px 2px rgba(0,0,0,.04); }
.card h3, .card h4 { margin-top: 0; }
.row { display: flex; gap: 10px; align-items: center; flex-wrap: wrap; }
.split { display: grid; gap: 16px; grid-template-columns: minmax(0, 1fr) 280px; }
.form { display: grid; gap: 12px; max-width: 480px; }
.form input, .form select, .form textarea { width: 100%; border: 1px solid #ccd4df; border-radius: 6px; padding: 10px; }
.btn { border: 0; border-radius: 6px; padding: 10px 12px; background: #0d6efd; color: #fff; cursor: pointer; }
.btn.secondary { background: #5c667a; }
.btn.ghost { background: #eef3f8; color: #14304d; }
.muted { color: #687386; }
.pill { display: inline-block; border-radius: 999px; padding: 3px 8px; font-size: 12px; background: #e9edf3; color: #40506a; }
.option { display: block; width: 100%; margin: 8px 0; border: 1px solid #d8dee8; border-radius: 8px; padding: 12px; background: #fff; text-align: left; color: #172033; }
.option.active { border-color: #0d6efd; background: #eef5ff; }
.option.correct { border-color: #249653; background: #e7f7ee; }
.option.wrong { border-color: #c0392b; background: #fdecec; }
table { width: 100%; border-collapse: collapse; font-size: 14px; }
th, td { border-bottom: 1px solid #e7ebf0; padding: 10px; text-align: left; vertical-align: top; }
@media (max-width: 760px) {
  .topbar { align-items: flex-start; flex-direction: column; }
  .toolbar a, .toolbar button, .toolbar select { padding: 7px 8px; font-size: 13px; }
  .split { grid-template-columns: 1fr; }
  th, td { padding: 8px 6px; font-size: 12px; }
}
```

文件：admin/package.json

> 职责：项目运行、构建或说明配置。
> 模块：后台服务与数据层

```
{
  "name": "school-chinese-exam-practice-admin",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "nodemon src/server.js",
    "start": "node src/server.js",
    "db:init": "node src/utils/initDb.js",
    "seed:users": "node src/utils/seedUsers.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "dotenv": "^16.4.7",
    "express": "^4.21.2",
    "jsonwebtoken": "^9.0.2",
    "mysql2": "^3.11.5"
  },
  "devDependencies": {
    "nodemon": "^3.1.9"
  }
}
```

文件：fronter/package.json

> 职责：项目运行、构建或说明配置。
> 模块：前台 H5 页面层

```
{
  "name": "school-chinese-exam-practice-fronter",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite --host 0.0.0.0",
    "build": "vite build",
    "preview": "vite preview --host 0.0.0.0"
  },
  "dependencies": {
    "@vitejs/plugin-vue": "^5.2.1",
    "vite": "^6.0.3",
    "vue": "^3.5.13",
    "vue-router": "^4.5.0"
  },
  "devDependencies": {}
}
```

文件：.gitignore

> 职责：项目运行、构建或说明配置。
> 模块：项目配置与说明

```
node_modules/
dist/
logs/
.tmp/
.playwright-mcp/

.DS_Store
*.log
*.tmp

admin/logs/
fronter/dist/
```

文件：README.md

> 职责：项目运行、构建或说明配置。
> 模块：项目配置与说明

````
# 马来西亚留学生汉语练习平台

一个基于 Vue3、Node.js/Express 和 MySQL 的三语汉语练习平台 MVP，支持中文、英文、马来语。

## 功能

- 学生 H5：注册登录、练习列表、答题提交、成绩记录、错题本、个人中心
- 管理台：学生列表、题库列表、练习/试卷列表、成绩记录、数据看板
- 题库：等级、分类、题目、选项、解析全部三语
- 数据库：MySQL，建表和演示题库位于 `admin/sql`

## 启动

```bash
cd admin
npm install
npm run db:init
npm run seed:users
npm run dev
```

```bash
cd fronter
npm install
npm run dev
```

默认账号：

- 管理员：`admin / ***`
- 学生：`student / ***`

默认 API 地址为 `http://***/api`。如需修改，前端可设置 `VITE_API_BASE`。
````